Classic planning with PDDL

Aleksandra Słomska
Zofia Narloch

March 16, 2026

1 Exercise 1.1: Emergency Service Logistics, Initial Release

In the first exercise, a domain and two small problems were created. To present the outcome of each problem, plan files have been attached. In both problems people are in the same room.

```
(define (problem problem1) (:domain p1)
(:objects
    drone1 - drone
    crate1 - crate
    person1 - person
    room1 depot - location
    food medicine - content
    left right - arm
)
(:init
    (person-at person1 room1)
    (crate-at crate1 depot)
    (content-crate food crate1)
    (drone-at drone1 depot)
    (empty-arm drone1 left)
    (empty-arm drone1 right)
)

(:goal (and
   (has-content-person person1 food)
))

)
```

```
( PICK-UP-CRATE CRATE1 DEPOT DRONE1 RIGHT )
( DRONE-MOVE DEPOT ROOM1 DRONE1 )
( DROP-CRATE ROOM1 PERSON1 CRATE1 RIGHT FOOD DRONE1 )
```

Listing 1: First problem and the outcome - one person and one box

```
(define (problem problem2) (:domain p1)
(:objects
    drone1 - drone
    crate1 - crate
    crate2 - crate
    crate3 - crate
    person1 - person
    person2 - person
    room1 depot - location
    food medicine - content
    left right - arm
)

(:init
    (person-at person1 room1)
    (person-at person2 room1)
    (crate-at crate1 depot)
    (crate-at crate2 depot)
    (crate-at crate3 depot)
    (content-crate food crate1)
    (content-crate medicine crate2)
    (content-crate food crate3)
    (drone-at drone1 depot)
    (empty-arm drone1 left)
    (empty-arm drone1 right)
)

(:goal (and
   (has-content-person person1 food)
   (has-content-person person2 medicine)
))

)
```

```
( PICK-UP-CRATE CRATE1 DEPOT DRONE1 RIGHT )
( PICK-UP-CRATE CRATE2 DEPOT DRONE1 LEFT )
( DRONE-MOVE DEPOT ROOM1 DRONE1 )
( DROP-CRATE ROOM1 PERSON1 CRATE1 RIGHT FOOD DRONE1 )
( DROP-CRATE ROOM1 PERSON2 CRATE2 LEFT MEDICINE DRONE1 )
```

Listing 2: Outcome of the second problem - two people and three boxes

2 Exercise 1.2: Problem Builder in Python

In the second exercise, the performance of the FF planner was tested. We created a Python script that runs the generate-problems.py algorithm, executing the generated problems until the execution time exceeded one minute. The problem size was incremented by two after each execution. Maximum size within one minute reached 45.

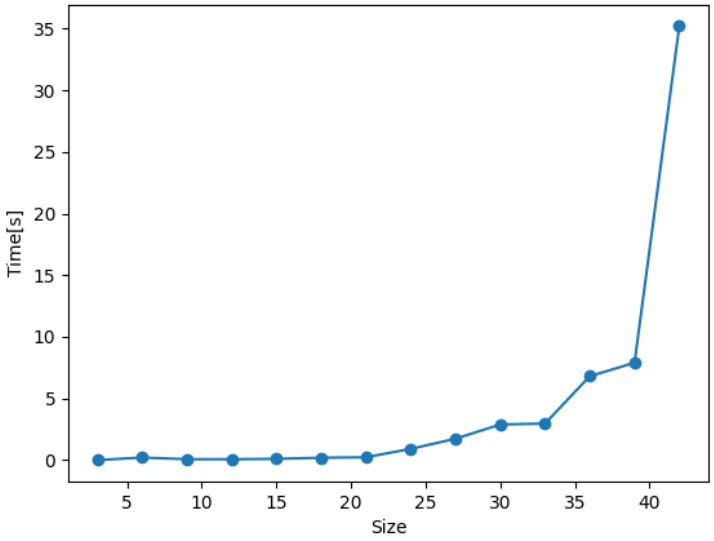

Figure 1: Size of the problem and the time required to find a solution in the tests

3 Exercise 1.3: Performance Comparison of Search Algorithms and Heuristics

3.1 First Part

Algorithm	Heuristic	Largest Solved Size	Time (s)	Plan Length	Optimal
BFS	None	7	24.59	22	Yes
IDS	None	3	0.71	10	Yes
A*	hMAX	5	9.95	17	Yes
GBFS	hMAX	7	10.94	26	No

Table 1: Performance Comparison of Search Algorithms and Heuristics (Exercise 1.3.1)

The results of the performance of every algorithm show that only GBFS with heuristic hMAX is not optimal. The reason for that is the longest plan length. This is because GBFS relies on the heuristic to rush toward the goal quickly rather than exhaustively searching for the perfect path. The same size of the problem was solved by the BFS algorithm without any heuristics. However, it took more time, because it blindly checks every possible combination of moves. On the other hand, the A* algorithm only reached size 5 because the calculation of the hMAX heuristic for every node caused it to time out. Algorithm IDS solved the smallest problem of them all. IDS intentionally wipes its memory and starts over every time it increases its search depth, so it concentrates on memory efficiency.

In this part every action in domain file has the same cost, that is why algorithms like BFS guarantee the shortest path by expanding all nodes at the current search depth before moving deeper. Secondly, at every step, the drone has many valid moves, which creates many possibilities, which can cause problems for some algorithms like BFS.

3.2 Second Part

Algorithm	**Heuristic**	**Time (s)**	**Plan Length**
GBFS	NONE	0.37	26
GBFS	hMAX	10.68	26
GBFS	hFF	0.28	26
GBFS	hADD	0.35	27
GBFS	LANDMARK	0.29	28
EHC	NONE	0.31	28
EHC	hMAX	Timeout	$>$ 60s
EHC	hFF	0.39	28
EHC	hADD	0.44	28
EHC	LANDMARK	0.27	31

Table 2: Performance Comparison of GBFS and EHC with Various Heuristics (Exercise 1.3.2)

Based on the results from Exercise 1.3.1, the largest problem that Greedy Best-First Search (GBFS) could handle within the 1-minute time limit was size 7. This problem size was generated and tested for the algorithms GBFS and EHC with heuristics hMAX, hFF, hADD and Landmark. Only algorithm EHC with heuristic hMAX exceeded the time limit. The GBFS solved a size 7 problem with hMAX in under 11 seconds.

EHC is more memory-efficient than GBFS, since it discards alternative paths once it makes a choice. It causes the algorithm to be fast and use minimal memory. However, it causes problems when the chosen path reaches dead-end, since it can't go back. The GBFS keeps all values in memory, so it can return if the path is unpromising, but at cost of taking much more memory.

The FF planer intelligently combines the strengths of both algorithms to maximize speed and reliability. The FF planner primarily relies on EHC paired with the hFF heuristic. However, if EHC gets trapped in a dead-end, the planner automatically changes to GBFS. The result of that is the planner can backtrack out of the dead-end, guaranteeing that a solution will eventually be found.

3.3 Third Part

Algorithm	Heuristic	Time (s)	Plan Length
A*	NONE	6.51	17
BFS	NONE	0.77	17
IDS	NONE	Timeout	> 60s
A*	hMAX	9.18	17
A*	LMCUT	47.76	17

Table 3: Performance of Optimal Planners and Admissible Heuristics (Exercise 1.3.3)

An admissible heuristic is one that never overestimates the true cost to reach the goal, guaranteeing that an algorithm like A* will find an optimal solution. Among the heuristics implemented in Pyperplan, hMAX and LMCUT (Landmark-Cut) are admissible.

The results show that almost every algorithm has a plan length equal to 17. The best-performing algorithm was BFS without any heuristics. The reason for that is the same cost for every action. Moreover, adding the heuristics to algorithm A* resulted in exceeding the time by about 1.4 times for hMAX and over 7 times for LMCUT. It is because of the necessity to calculate these complex heuristics at every node.

Finally, IDS timed out because a plan length of 17 steps requires it to wipe its memory and completely regenerate the search tree from scratch 17 times, which causes it to exceed the 1-minute limit.

4 Exercise 2.1: Problem Builder in Python

In this exercise new actions have been added.
PUT-CRATE-IN-CARRIER - drone puts a crate in one of the spots of the carrier
MOVE-CARRIER - drone moves with carrier to a new location
PICK-AND-DROP - drone picks one of the crates from the carrier and gives it to a person

```
(define (problem drone_problem_d1_r1_l2_p2_c2_g2_ct2)
(:domain p2)
(:objects
```

```
  drone1 - drone
  depot - location
  loc1 - location
  loc2 - location
  crate1 - crate
  crate2 - crate
  food - content
  medicine - content
  person1 - person
  person2 - person
  carrier1 - carrier
  N0 - num
  N1 - num
  N2 - num
  N3 - num
  N4 - num
)
(:init
  (= (total-cost) 0)
  (= (fly-cost depot depot) 1)
  (= (fly-cost depot loc1) 223)
  (= (fly-cost depot loc2) 68)
  (= (fly-cost loc1 depot) 223)
  (= (fly-cost loc1 loc1) 1)
  (= (fly-cost loc1 loc2) 189)
  (= (fly-cost loc2 depot) 68)
  (= (fly-cost loc2 loc1) 189)
  (= (fly-cost loc2 loc2) 1)
  (drone-at drone1 depot)
  (crate-at crate1 depot)
  (crate-at crate2 depot)
  (carrier-at carrier1 depot)
  (crates-in-carrier carrier1 N0)
  (content-crate food crate1)
  (content-crate medicine crate2)
  (person-at person1 loc1)
  (person-at person2 loc2)
  (next-num N0 N1)
  (next-num N1 N2)
  (next-num N2 N3)
  (next-num N3 N4)
)
(:goal (and
  (drone-at drone1 depot)
  (has-content-person person2 food)
  (has-content-person person2 medicine)
))
(:metric minimize (total-cost))
)
```

```
( PUT-CRATE-IN-CARRIER CRATE1 DEPOT DRONE1 CARRIER1 N0 N1 )
( PUT-CRATE-IN-CARRIER CRATE2 DEPOT DRONE1 CARRIER1 N1 N2 )
( MOVE-CARRIER DEPOT LOC2 DRONE1 CARRIER1 )
( PICK-AND-DROP CRATE1 LOC2 DRONE1 CARRIER1 PERSON2 FOOD N2 N1 )
( PICK-AND-DROP CRATE2 LOC2 DRONE1 CARRIER1 PERSON2 MEDICINE N1 N0
   )
```

```
6 ( MOVE-CARRIER LOC2 DEPOT DRONE1 CARRIER1 )
```

Listing 3: Outcome of the second problem - two people and three boxes

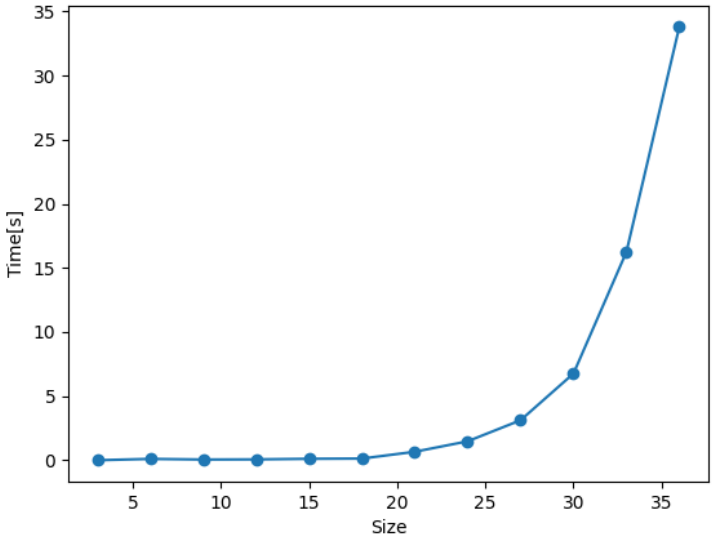

Figure 2: Size of the problem and the time required to find a solution in the tests

Algorithm	**Heuristic**	**Time P1 (s)**	**Time P2 (s)**	**Plan P1**	**Plan P2**
GBFS	NONE	0.37	0.62	26	23
GBFS	hMAX	10.68	Timeout	26	> 60s
GBFS	hFF	0.28	0.62	26	23
GBFS	hADD	0.35	0.54	27	26
GBFS	LANDMARK	0.29	0.42	28	28
EHC	NONE	0.31	0.54	28	23
EHC	hMAX	Timeout	Timeout	> 60s	> 60s
EHC	hFF	0.39	0.47	28	23
EHC	hADD	0.44	1.18	28	30
EHC	LANDMARK	0.27	0.43	31	31

Table 4: Performance Comparison of GBFS and EHC

Algorithm	Heuristic	Time P1 (s)	Time P2 (s)	Plan P1	Plan P2
A*	NONE	6.51	4.71	17	14
BFS	NONE	0.77	0.64	17	14
IDS	NONE	Timeout	Timeout	$>$ 60s	$>$ 60s
A*	hMAX	9.18	8.59	17	14
A*	LMCUT	47.76	42.49	17	14

Table 5: Performance of Optimal Planners

By adding the carriers the plan length was improved. The reason for that is the drone does not have to go back to depot to take more crates, he is able to take more at once. The time also improved in some cases especially in algorithm A with or without heuristics and BFS. In algorithms GBFS and EHC time has improved with most of the heuristics. This significant improvement in execution time is a result of modeling the carrier capacity using numerical tracking rather than slots.

5 Exercise 2.2: Problem Builder in Python

5.1 First Part - satisficing planners

Planner	Alias	Max problem size	Cost(biggest problem)	Cost(size 5)
Metric-ff	NONE	11 (time 0.1s)	2476	1472
Fast Downward	lama-first	125	34490	1472
Fast Downward	seq-sat-fd-autotune-2	7	637	723
Fast Downward	seq-sat-fdss-2	12	1818	723

Metrif-ff has been measured only up to problem size 11. Bigger problems did not execute, most probably because of Metric-ff problem size limitations. Unlike optimal planners, satisficing planners do not guarantee the lowest possible cost. They use Greedy Best-First Search to find a "good enough" valid plan before the time limit expires. By trading mathematical perfection for raw speed, these planners can scale to much larger and more complex real-world problems.

- **Metric-FF** - uses Enforced Hill-Climbing and the Fast-Forward heuristic. As shown in the snippet below (problem size 5) it prioritizes speed over quality, evaluating only 58 states. This blind sprinting results in highly unoptimized routing: for example, instead of maximizing carrier capacity, it makes multiple redundant round-trips to the exact same location, resulting in a cost of 1472 for size 5 (double the optimal cost).

```
ff: search configuration is Enforced Hill-Climbing, then A*
    epsilon with weight 5.
...
Cueing down from goal distance:    11 into depth [1]
...
        8: PICK-AND-DROP CRATE3 LOC1 DRONE1 CARRIER1 PERSON3
    FOOD N1 N0
```

```
        9: MOVE-CARRIER LOC1 DEPOT DRONE1 CARRIER1
...
plan cost: 1472.000000
time spent: ... 0.00 seconds searching, evaluating 58 states,
    to a max depth of 4

```

- **lama-first** - to maximize speed, it temporarily sets all action costs to 1. It stops immediately after finding first valid plan. This lack of optimization allows it solve big problems within short time (as shown in the snippet below taken from the output of a problem size 125, it generated over 3 million states), but the cost is higher than when that of other alises.

```
[t=5.824840s] Discovered 394 landmarks...
...
[t=44.423614s] Solution found!
[t=44.423742s] Plan length: 480 step(s).
[t=44.423742s] Plan cost: 34490
[t=44.423742s] Generated 3124908 state(s).

```

- **seq-sat-fd-autotune-2** - this planner uses multiple heuristics at once (Causal Graph, Context-Enhanced Additive, Fast-Forward, and Goal-count) through an alternating queue. It finds a quick base plan, then uses the remaining time to lower the acceptable cost boundary and search again. This loops until time runs out or the space is exhausted, optimizing the cost but limiting its maximum scale due to calculating four different heuristics for a single state.

```
args: [... '--search', '... lazy_greedy([cg(), cea(), ff(),
    goalcount()] ...)' ...]
[t=0.015s] Initializing causal graph heuristic...
[t=0.016s] Initializing context-enhanced additive heuristic...
[t=0.017s] Initializing FF heuristic...
[t=0.018s] Initializing goalcount heuristic...
...
[t=0.016748s] Solution found!
[t=0.017024s] Plan cost: 1860
...
[t=0.036964s] Solution found!
[t=0.037225s] Plan cost: 1558
...
[t=9.347671s] Solution found!
[t=9.347953s] Plan cost: 637
[t=43.583179s] Completely explored state space -- no solution!

```

- **seq-sat-fdss-2** -it splits the 60-second time limit across different search algorithms. As seen in the size 12 problem, first: completely ignoring real flight costs to get a fast plan on using the Fast-Forward heuristic (cost 3064). It then aborts, turns real costs back on (cost type=normal), and uses the previous 3064 cost as a boundary to find a better route (cost

2286). Finally, it switches to an Iterated Weighted Search, slowly dropping the heuristic weight (weight(h,5), weight(h,3), etc.) to squeeze the final cost down to 1818 before the system timeout physically cuts it off.

```
args: [... '--search', 'let(h, ff(...),eager_greedy([h]...
    cost_type=one,bound=infinity))' ...]
[t=0.010s] Initializing FF heuristic...
[t=0.046154s] Solution found!
[t=0.046154s] Plan cost: 3064
...
Switch to real costs and repeat last run.
args: [... '--search', 'let(h, ff(...),eager_greedy([h]...
    cost_type=normal,bound=3064))' ...]
[t=0.023751s] Solution found!
[t=0.023751s] Plan cost: 2286
...
Abort portfolio and run final config.
args: [... '--search', 'let(h, ff(...),iterated([eager(single(
    sum([g(),weight(h,5)])))) ... repeat_last=true))' ...]
[t=2.762474s] Solution found!
[t=2.762474s] Plan cost: 1818
...
caught signal 24 -- exiting

```

5.2 Second Part - optimal planners

Planner	Alias	Max problem size	Cost(biggest problem)	Cost(size 5)
Fast Downward	seq-opt-lmcut	7	637	723
Fast Downward	seq-opt-bjolp	8	783	723
Fast Downward	seq-opt-fdss-2	11	927	723

All of the optimal planners returned the same optimal cost for the same problem. They all used A* search, which means plans can not finish executing untill they find an optimal solution.

- seq-opt-lmcut - builds graphs and makes mathematical cuts to find the best solution. Cuts are costly in time, so planner managed to find solution to the smallest size of a problem out of all optimal planners. Cost for size 7 is smaller than cost for size 5 due to random values of cost of actions and random goals by the algorithm to each problem.

```
[t=0.005246s] Initial heuristic value for lmcut: 235
...
[t=0.326000s] New best heuristic value for lmcut: 131
[t=0.326045s] New best heuristic value for lmcut: 0
[t=0.326094s] Solution found!

```

- seq-opt-bjopl - scans the problem to discover the landmarks (actions that must happen). Finding the landmarks takes most of the time of the search, then evaluating teh steps during search is faster.

```
[t=0.005181s] Merging 2 landmark graphs
[t=0.006838s] Discovered 23 landmarks...
[t=0.007555s] Initial heuristic value for
    landmark_cost_partitioning: 190
...
[t=0.075162s] New best heuristic value for
    landmark_cost_partitioning: 0
[t=0.099509s] Solution found!
[t=0.099524s] Actual search time: 0.091956s

```

- seq-opt-fdss-2 - uses precalculated table to search for the solution - merge and shrink algorithm, which is faster than complex math done by other planners.

```
[t=0.007845s] Running merge-and-shrink algorithm...
[t=0.117385s] Done initializing merge-and-shrink heuristic.
...
[t=0.326045s] New best heuristic value for merge_and_shrink: 0
[t=0.326118s] Actual search time: 0.320856s

```

6 Exercise 3: Concurrent actions in emergency management

Drones	Max problem size	Mode	Processing Time [s]	Number of steps	Total Duration
1	5	Quality	4.76	23	2042.00
		Speed	0.14	23	1850.00
2	3	Quality	2.76	13	638.00
		Speed	0.28	12	530.00
3	3	Quality	84.26	13	414.00
		Speed	1.25	13	778.00
4	3	Quality	19.02	16	510.00
		Speed	2.66	17	1142.00
5	3	Quality	315.97	12	379.00
		Speed	14.39	14	409.00

Table 6: Comparison of LPG-TD planner performance in Quality and Speed modes.